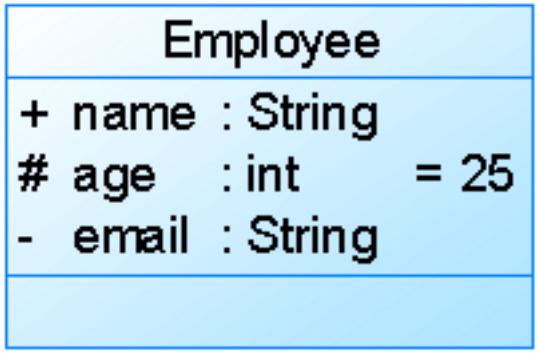
类图

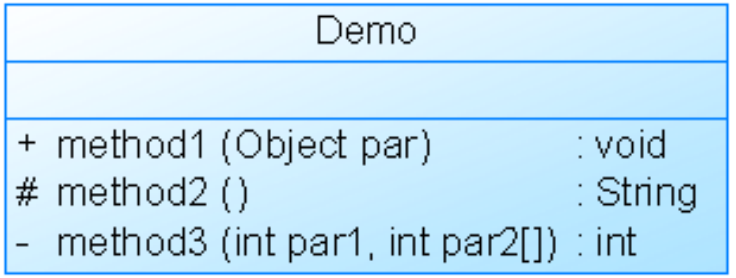
- 类与类图
  - ✓ 类(Class)封装了数据和行为，是面向对象的重要组成部分，它是具有相同属性、操作、关系的对象集合的总称。
  - ✓ 在系统中，每个类具有一定的职责，职责指的是类所担任的任务，即类要完成什么样的功能，要承担什么样的义务。一个类可以有多种职责，设计得好的类一般只有一种职责，在定义类的时候，将类的职责分解成为类的属性和操作（即方法）。
  - ✓ 类的属性即类的数据职责，类的操作即类的行为职责。





类图

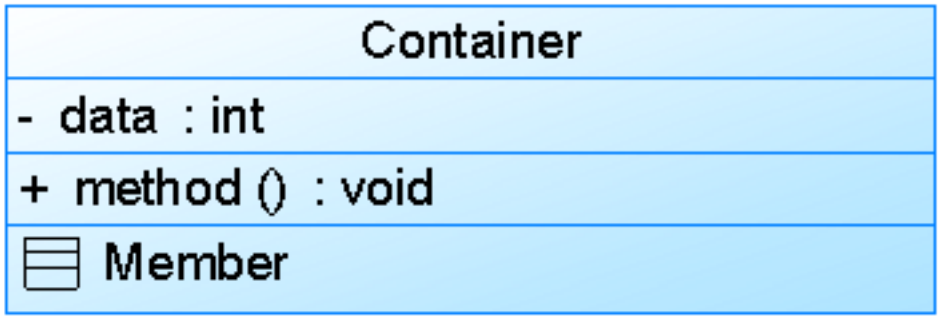




- 类与类图
  - ✓ 在**UML**类图中，类一般由三部分组成：
    - 类名：每个类都必须有一个名字，类名是一个字符串。
    - 属性(Attributes)：属性是指类的性质，即类的成员变量。类可以有任意多个属性，也可以没有属性。

可见性 名称:类型 [= 默认值]

    - 操作(Operations)：操作是类的任意一个实例对象都可以使用的行为，操作是类的成员方法。



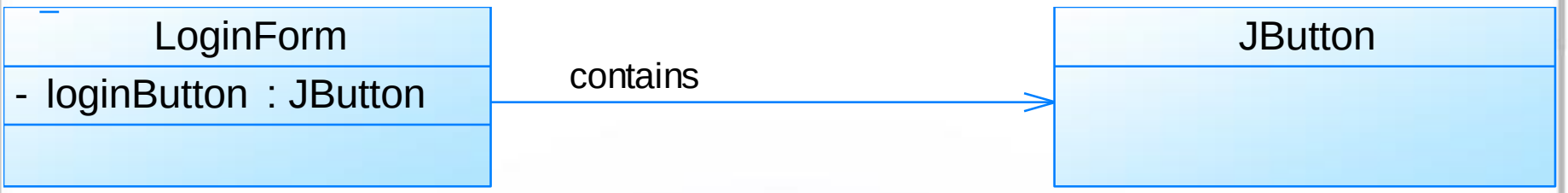
类图

- 类之间的关系
  - ✓关联关系
    - 关联关系(Association)是类与类之间最常用的一种关系，它是一种结构化关系，用于表示一类对象与另一类对象之间有联系。
    - 在UML类图中，用实线连接有关联的对象所对应的类，在使用Java、C#和C++等编程语言实现关联关系时，通常将一个类的对象作为另一个类的属性。
    - 在使用类图表示关联关系时可以在关联线上标注角色名。



类图

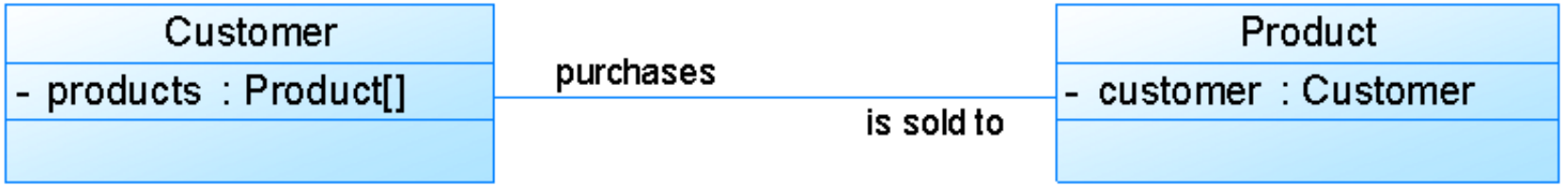
- 类之间的关系
  - ✓ 关联关系




```
public class LoginForm
{
   private JButton loginButton;
    ……
}
public class JButton
{
   ……
}
```

类图

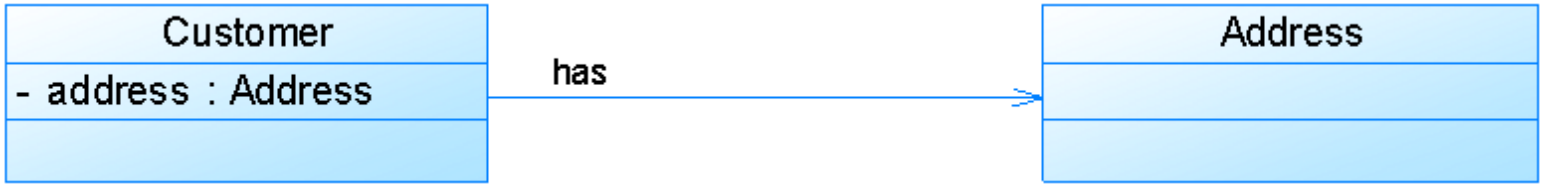
- 类之间的关系
  - ✓双向关联
    - 默认情况下，关联是双向的。




```
public class Customer
{
   private Product[] products;
   ……
}
public class Product
{
   private Customer customer;
   ……
}
```

类图

- 类之间的关系
  - ✓ 单向关联
    - 类的关联关系也可以是单向的，单向关联用带箭头的实线表示。



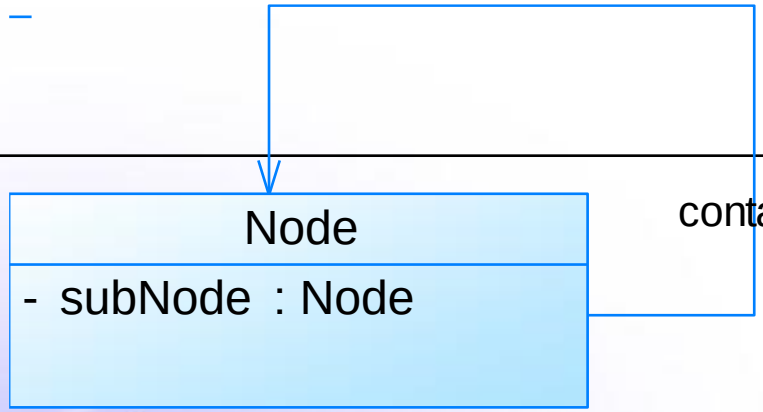

```
public class Customer
{
   private Address address;
   ……
}

public class Address
{
   ……
}
```

类图

- 类之间的关系
  - ✓ 自关联
    - 在系统中可能会存在一些类的属性对象类型为该类本身，这种特殊的关联关系称为自关联。

```
public class Node
{
    private Node subNode;
    ……
}
```



类图

关联关系

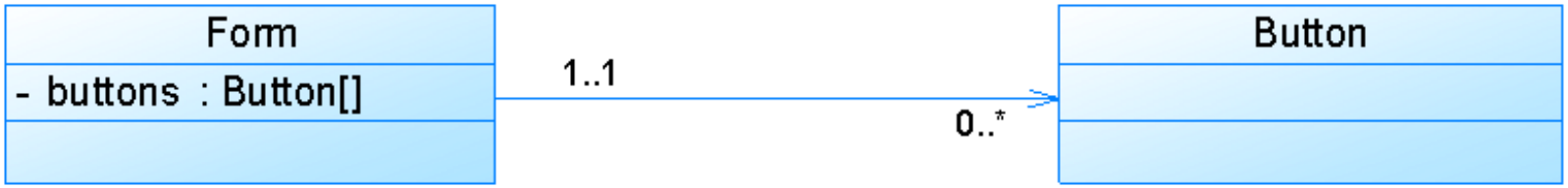
- 多重性关联：多重性关联关系又称为重数性(Multiplicity)关联关系，表示两个关联对象在数量上的对应关系。在UML中，对象之间的多重性可以直接在关联直线上用一个数字或一个数字范围表示

表示方式	多重性说明
1..1	表示另一个类的一个对象只与该类的一个对象有关系
0..*	表示另一个类的一个对象与该类的零个或多个对象有关系
1..*	表示另一个类的一个对象与该类的一个或多个对象有关系
0..1	表示另一个类的一个对象没有或只与该类的一个对象有关系
m..n	表示另一个类的一个对象与该类最少m，最多n个对象有关系 (m≤n)



类图

- 类之间的关系
  - ✓多重性关联



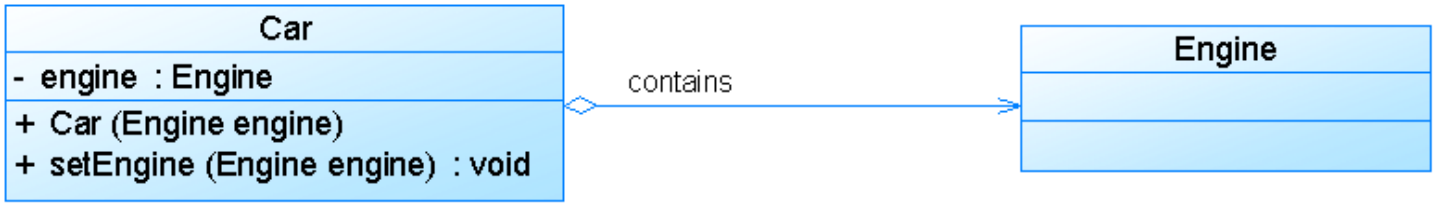

```
public class Form
{
    private Button buttons[];
    ……
}
public class Button
{
    …
}
```

类图

- **关联关系**
  - ✓聚合关系
    - 聚合(Aggregation)关系表示整体与部分的关系
    - 在聚合关系中，成员对象是整体对象的一部分，但是成员对象可以脱离整体对象独立存在
    - 在UML中，聚合关系用带空心菱形的直线表示



类图



- 类之间的关系
  - ✓聚合关系

```
public class Car
{
   private Engine engine;
   public Car(Engine engine)
   {
      this.engine = engine;
   }

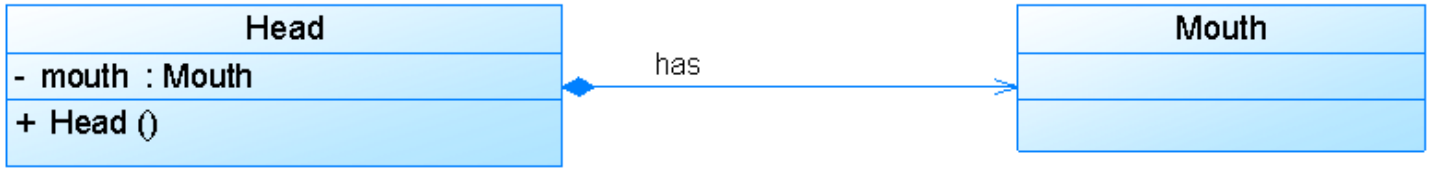
   public void setEngine(Engine engine)
   {
      this.engine = engine;
   }
    ……
}
public class Engine
{
   ……
}
```

类图

- **关联关系**
  - ✓组合关系
    - 组合(Composition)关系也表示类之间整体和部分的关系，但是在组合关系中整体对象可以控制成员对象的生命周期，一旦整体对象不存在，成员对象也将不存在
    - 成员对象与整体对象之间具有同生共死的关系
    - 在UML中，组合关系用带实心菱形的直线表示



类图



- 类之间的关系
  - ✓组合关系

```
public class Head
{
   private Mouth mouth;
   public Head()
   {
         mouth = new Mouth();
   }
    ……
}

public class Mouth
{
   ……
}
```

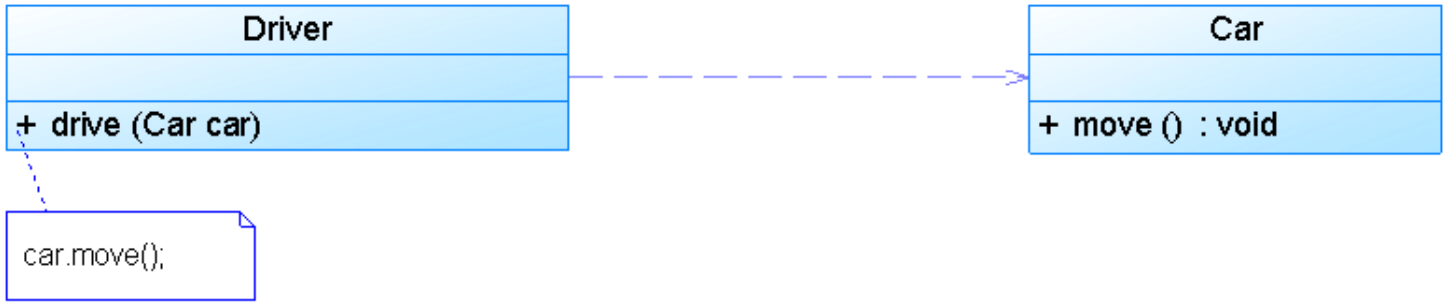
类图

- 依赖关系
  - ✓ 依赖(Dependency)关系是一种使用关系，特定事物的改变有可能会影响到使用该事物的其他事物，在需要表示一个事物使用另一个事物时使用依赖关系。
  - ✓ 大多数情况下，依赖关系体现在某个类的方法使用另一个类的对象作为参数。
  - ✓ 在UML中，依赖关系用带箭头的虚线表示，由依赖的一方指向被依赖的一方。





类图



- 类之间的关系
  - ✓依赖关系

```
public class Driver
{
   public void drive(Car car)
   {
      car.move();
   }
   ……
}
public class Car
{
   public void move()
   {
      ......
   }
   ……
}
```

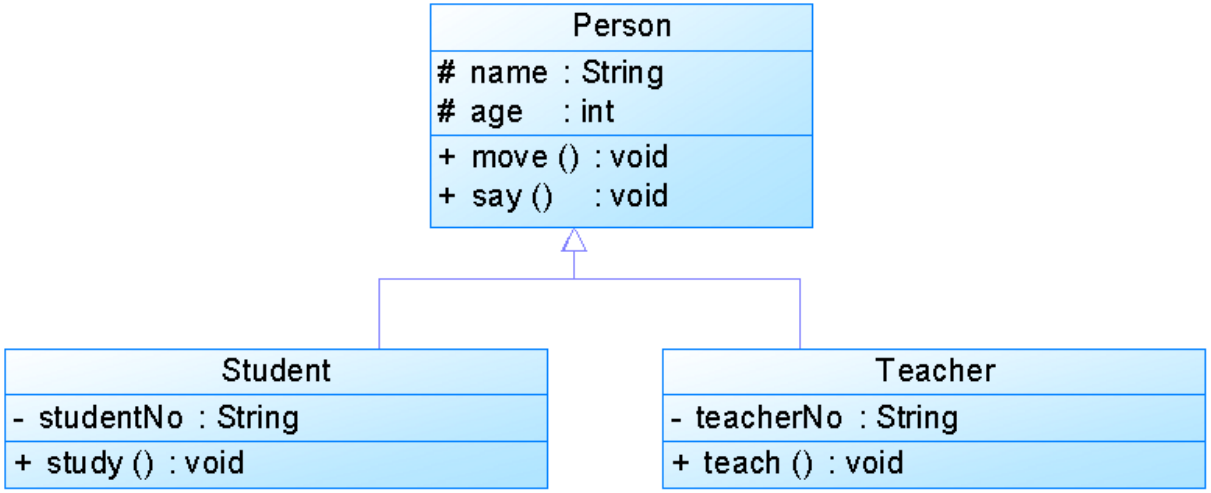
类图

- 类之间的关系
  - ✓泛化关系
    - 泛化关系(Generalization)也就是继承关系，也称为“is-a-kind-of”关系，泛化关系用于描述父类与子类之间的关系，父类又称作基类或超类，子类又称作派生类。在UML中，泛化关系用带空心三角形的直线来表示。
    - 在代码实现时，使用面向对象的继承机制来实现泛化关系，如在Java语言中使用extends关键字、在C++/C#中使用冒号“：”来实现。



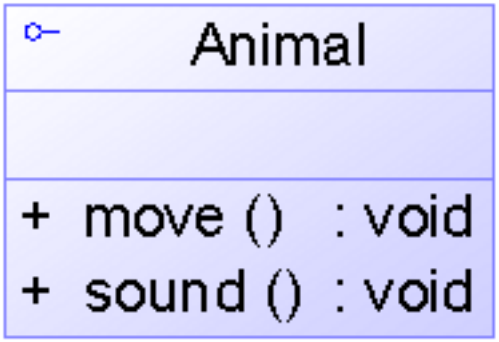
类图

- 类之间的关系
  - ✓泛化关系



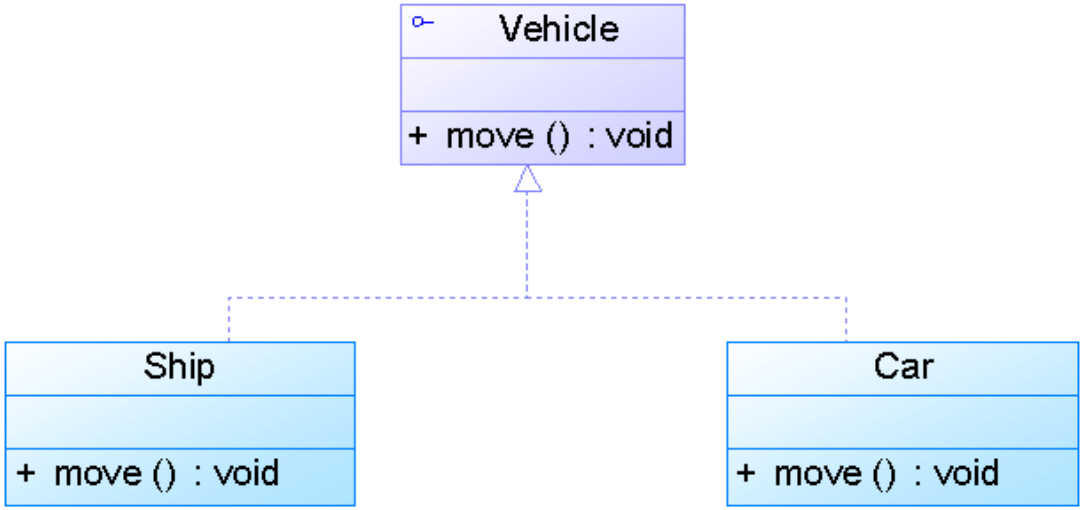

```
public class Person
{
    protected String name;
    protected int age;
    public void move()
    {
        ……
    }
    public void say()
    {
        ……
    }
}
public class Student extends Person
{
    private String studentNo;
    public void study()
    {
        ……
    }
}
```

类图



- 类之间的关系
  - ✓接口与实现关系
    - 接口之间也可以有与类之间关系类似的继承关系和依赖关系，但是接口和类之间还存在一种实现关系(Realization)，在这种关系中，类实现了接口，类中的操作实现了接口中所声明的操作。在UML中，类与接口之间的实现关系用带空心三角形的虚线来表示。

类图



- 类之间的关系
  - ✓接口与实现关系

```
public interface Vehicle
{
    public void move();
}
public class Ship implements Vehicle
{
    public void move()
    {
    ……
    }
}
public class Car implements Vehicle
{
    public void move()
    {
    ……
    }
}
```

类图

- 注释(Comment)

